Projeto Detalhado de Sistemas de Software: Plataforma Inteligente de Monitoramento de Uso e Cobertura do Solo (PIMUCS)

Lorran de Araújo Durães Soares[1]

[1]Laboratório Nacional de Computação Científica (LNCC)
Petrópolis - RJ - Brasil

1. Introdução

Este trabalho apresenta o projeto detalhado da **Plataforma Inteligente de Monitoramento de Uso e Cobertura do Solo (PIMUCS)**, dando continuidade à especificação arquitetural desenvolvida no primeiro trabalho. A plataforma tem como objetivo integrar a obtenção periódica de imagens georreferenciadas, o processamento desses dados, a atualização de um modelo de inteligência artificial, a geração de mapas de uso e cobertura do solo, sua disponibilização em ambiente web e o monitoramento de possíveis incompatibilidades com o zoneamento ambiental oficial.

O detalhamento concentra-se principalmente no módulo *IAProcessing*, responsável pelo fluxo automático de atualização dos mapas e pelo monitoramento ambiental. Para representar adequadamente suas interações, também são detalhadas classes pertencentes aos módulos de gerenciamento de dados, acesso controlado, comunicação com o sistema externo e representação das entidades manipuladas ao longo do processamento. A solução foi estruturada segundo o paradigma de orientação a objetos, e os protótipos apresentados neste trabalho foram implementados em Python.

Durante o projeto detalhado, algumas responsabilidades que estavam implícitas na arquitetura inicial foram explicitadas por meio de novos pacotes lógicos, abstrações e classes auxiliares. Essas alterações não modificam os requisitos ou objetivos funcionais definidos anteriormente, mas refinam a organização interna do sistema, reduzem o acoplamento entre responsabilidades distintas e tornam mais claras as dependências necessárias para sua futura implementação completa.

2. Especificação detalhada dos módulos selecionados

2.1. Diagramas de classes

O detalhamento concentra-se no módulo *IAProcessing*, responsável pela atualização automática dos mapas de uso e cobertura do solo e pelo monitoramento ambiental, incluindo também as classes de apoio necessárias a esse fluxo. Esse refinamento parte da visão de módulos definida anteriormente, na qual os elementos arquiteturais são organizados segundo suas responsabilidades e relações [Clements et al. 2003]. A Figura 1 apresenta o diagrama de classes resultante.

As classes estão agrupadas nos pacotes lógicos *IAProcessing*, *DataManagement*, *WebApplication*, *External* e *Models*. Essa organização preserva a separação de responsabilidades estabelecida na arquitetura e explicita as relações entre os elementos responsáveis pelo processamento, gerenciamento de dados, interação com sistemas externos e acesso dos usuários [Clements et al. 2003].

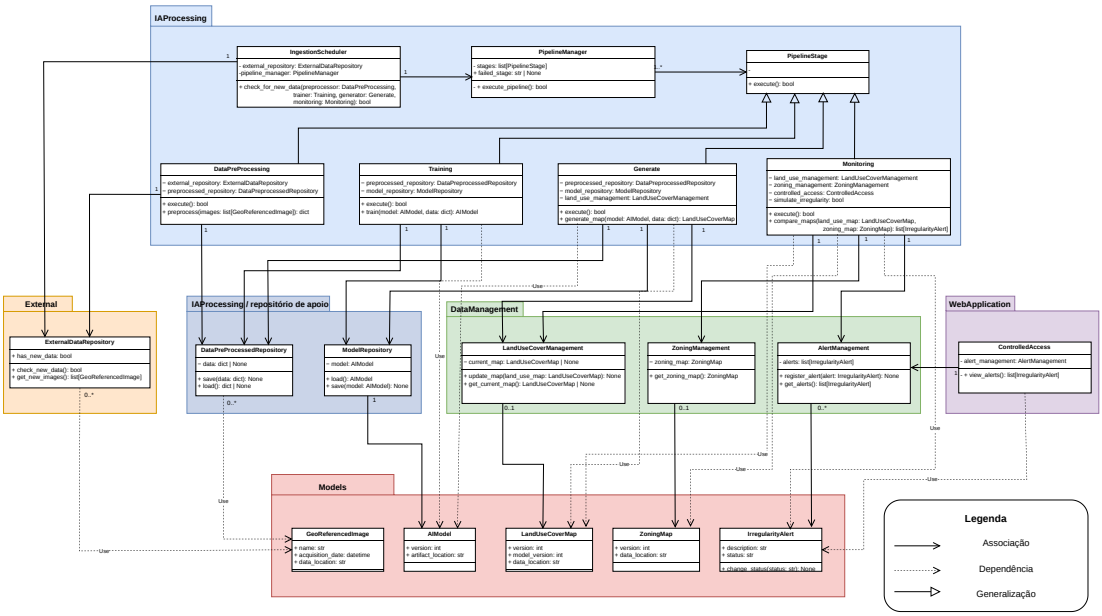


Figura 1. Diagrama de classes da PIMUCS, com detalhamento do módulo *IAProcessing* e das classes de apoio.

O pacote *External* explicita uma dependência já existente na arquitetura: o acesso a um repositório externo de imagens georreferenciadas. A classe *ExternalDataRepository* encapsula essa comunicação e pressupõe a existência de uma fonte capaz de informar a disponibilidade de novas imagens e fornecer seus dados. O *IngestionScheduler* utiliza `check_new_data()` para verificar a necessidade de atualização, enquanto *DataPreProcessing* utiliza `get_new_images()` para obter os objetos *GeoReferencedImage* correspondentes às novas imagens, sem precisar conhecer os mecanismos de acesso à fonte externa.

O pacote *Models* reúne *GeoReferencedImage*, *AIModel*, *LandUseCoverMap*, *ZoningMap* e *IrregularityAlert*, que representam os principais dados manipulados pela plataforma. Essas entidades também mantêm referências aos respectivos artefatos: *GeoReferencedImage*, *LandUseCoverMap* e *ZoningMap* utilizam `data_location`, enquanto *AIModel* utiliza `artifact_location`. Esses atributos apenas identificam a localização dos dados; sua leitura, escrita e gerenciamento permanecem sob responsabilidade das classes especializadas.

O fluxo automático é iniciado por *IngestionScheduler*, que mantém referências para *ExternalDataRepository* e *PipelineManager*. Sua operação `check_for_new_data()` consulta a fonte externa e aciona o pipeline apenas quando novos dados são identificados. O *PipelineManager*, por sua vez, mantém uma coleção ordenada de objetos *PipelineStage* e o atributo `failed_stage`. Sua operação `execute_pipeline()` percorre as etapas na ordem configurada e interrompe a execução quando uma delas informa falha.

PipelineStage é uma classe abstrata criada para definir o contrato comum `execute(): bool`. As classes *DataPreProcessing*, *Training*, *Generate* e *Monitoring* a especializam, permitindo que o *PipelineManager* execute etapas distintas por meio

da mesma interface. Essa abstração reduz o acoplamento e facilita a inclusão, remoção ou substituição de etapas, favorecendo a modificabilidade do sistema.

A classe *DataPreProcessing* mantém referências para *ExternalDataRepository* e *DataPreprocessedRepository*. Sua operação `execute()` obtém as novas imagens, aplica `preprocess()` e armazena o resultado. Na implementação completa, o pré-processamento deverá incluir atividades como validação, normalização, reprojeção, recorte e controle de qualidade. O *DataPreprocessedRepository* disponibiliza `save()` e `load()`, permitindo compartilhar os dados processados com as etapas seguintes; na implementação *dummy*, esses dados são mantidos de forma simplificada em memória.

A classe *Training* utiliza *DataPreprocessedRepository* e *ModelRepository*. A operação `execute()` recupera os dados e o modelo vigente, enquanto `train()` representa sua atualização. Sua execução pressupõe a disponibilidade de dados pré-processados e de um modelo armazenado. O *ModelRepository* encapsula o armazenamento e a recuperação dos objetos *AIModel*, evitando que as etapas conheçam diretamente o mecanismo físico de persistência.

A classe *Generate* utiliza os dados pré-processados e o modelo atualizado para produzir um *LandUseCoverMap* por meio de `generate_map()`. O mapa é encaminhado ao *LandUseCoverManagement*, responsável por sua atualização, consulta e, futuramente, pelo histórico das versões e acesso aos arquivos correspondentes.

A etapa *Monitoring* mantém referências para *LandUseCoverManagement*, *ZoningManagement* e *AlertManagement*. Ela obtém o mapa de uso e cobertura e o zoneamento vigente por meio de `get_current_map()` e `get_zoning_map()`, utilizando `compare_maps()` para identificar incompatibilidades. O *ZoningManagement* encapsula o acesso e a atualização do zoneamento oficial, enquanto *AlertManagement* registra e disponibiliza os objetos *IrregularityAlert* produzidos pelo monitoramento.

A classe *ControlledAccess*, pertencente a *WebApplication*, consulta os alertas por meio de *AlertManagement*. Dessa forma, *Monitoring* não depende diretamente da aplicação web. Na implementação completa, *ControlledAccess* também será responsável pelos mecanismos de autenticação, autorização e operações administrativas.

As relações apresentadas na Figura 1 seguem a notação UML [Object Management Group 2017], e distinguem associações, dependências e generalizações. As associações representam referências mantidas pelas classes; as dependências indicam usos pontuais, como parâmetros e retornos; e a generalização representa a especialização de *PipelineStage*. As multiplicidades indicam a quantidade de objetos envolvidos em cada relação.

Os repositórios físicos de mapas, zoneamento, modelos e dados intermediários não precisam aparecer como classes no diagrama, embora continuem fazendo parte da arquitetura. *LandUseCoverManagement*, *ZoningManagement*, *ModelRepository* e *DataPreprocessedRepository* encapsulam o acesso a esses recursos, podendo futuramente empregar mecanismos eficientes de leitura, escrita e acesso parcial aos dados sem que as demais classes conheçam esses detalhes.

2.1.1. Padrões e princípios de projeto empregados

O projeto emprega padrões de projeto com o objetivo de reduzir o acoplamento e separar responsabilidades recorrentes [Gamma et al. 1994]. O padrão *Repository* é utilizado conceitualmente em *ModelRepository* e *DataPreprocessedRepository*, isolando as etapas de processamento dos mecanismos de armazenamento. O mesmo princípio de encapsulamento do acesso aos dados é utilizado por *LandUseCoverManagement*, *ZoningManagement* e *AlertManagement*.

A classe *ExternalDataRepository* desempenha papel semelhante ao padrão *Facade*, oferecendo uma interface simplificada para o acesso ao sistema externo de sensoriamento remoto [Gamma et al. 1994]. Assim, as demais classes utilizam operações como `check_new_data()` e `get_new_images()` sem precisar conhecer detalhes de protocolos, APIs ou mecanismos de comunicação da fonte externa.

Além desses padrões, o projeto utiliza abstração e polimorfismo por meio de *PipelineStage*. As etapas concretas compartilham o contrato `execute()`, permitindo que o *PipelineManager* execute objetos de tipos distintos por meio de uma interface comum, favorecendo a modificabilidade do pipeline.

2.2. Diagramas de sequência

Foram detalhados os dois principais fluxos relacionados à atualização dos mapas e ao monitoramento ambiental. As Figuras 2 e 3 complementam o diagrama de classes ao representar temporalmente as interações entre os objetos, segundo a notação de diagramas de sequência da UML [Object Management Group 2017]. As setas contínuas representam chamadas de operações entre os objetos, enquanto as setas tracejadas representam seus respectivos retornos.

A Figura 2 apresenta o fluxo iniciado pelo *IngestionScheduler*, que consulta *ExternalDataRepository* por meio de `check_new_data()` e, quando existem novos dados, solicita ao *PipelineManager* a execução do pipeline.

O *PipelineManager* executa sequencialmente as instâncias concretas de *PipelineStage* por meio de `execute()`. *DataPreProcessing* obtém as novas imagens com `get_new_images()`, realiza o pré-processamento e armazena o resultado em *DataPreprocessedRepository*. Em seguida, *Training* recupera esses dados e o modelo vigente, executa `train()` e armazena a nova versão em *ModelRepository*. Por fim, *Generate* recupera os dados e o modelo atualizado, executa `generate_map()` e encaminha o novo *LandUseCoverMap* ao *LandUseCoverManagement*.

A Figura 3 apresenta a etapa de monitoramento. O *PipelineManager* executa *Monitoring*, que recupera o mapa vigente de *LandUseCoverManagement* e o zoneamento de *ZoningManagement*. A operação `compare_maps()` realiza a comparação entre os dois objetos. Caso seja identificada uma irregularidade, é criado um *IrregularityAlert* e realizado seu registro em *AlertManagement*; caso contrário, a etapa é encerrada sem criação de alerta. Sua posterior consulta ocorre por meio de *ControlledAccess*, em um fluxo independente da etapa de monitoramento.

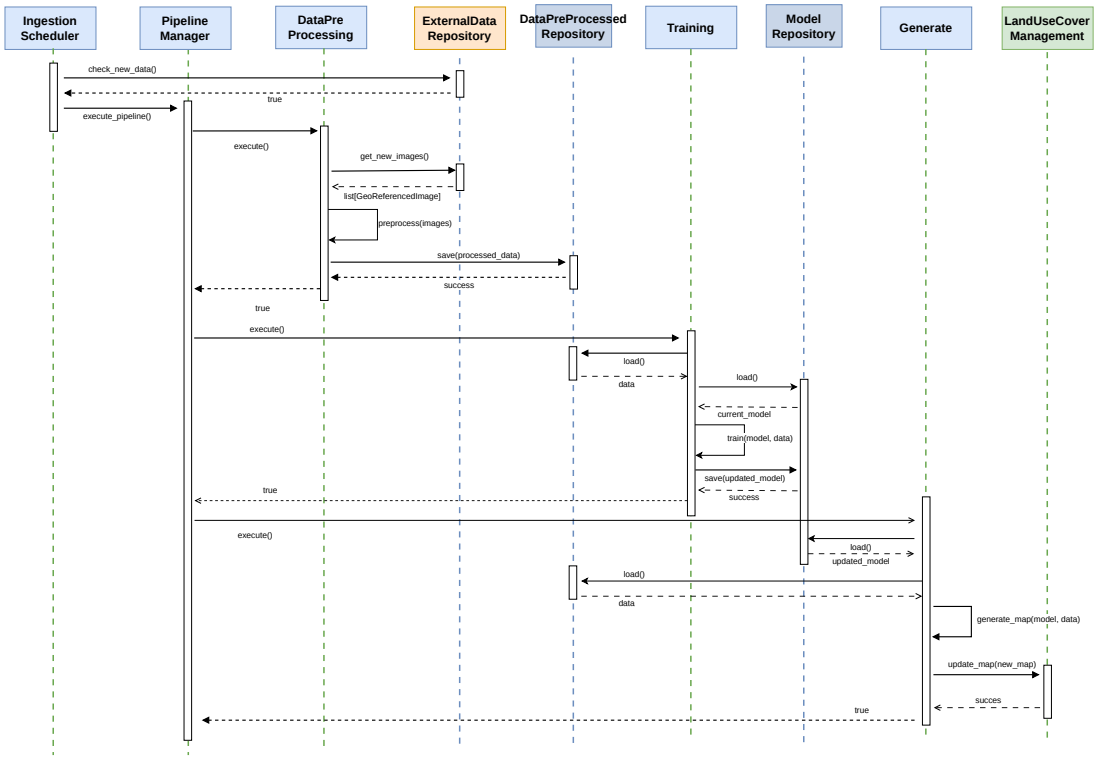


Figura 2. Diagrama de sequência do processo de atualização do mapa de uso e cobertura do solo.

2.3. Considerações sobre as especificações arquiteturais

O processo de detalhamento permitiu identificar elementos que estavam implícitos na arquitetura inicial e que precisaram ser explicitados para possibilitar uma implementação orientada a objetos. As principais alterações foram a criação dos pacotes lógicos *External* e *Models*, a introdução da abstração *PipelineStage* e a inclusão de *AlertManagement* entre o processamento e a aplicação web.

O pacote *External* não representa um novo requisito, mas explicita a dependência já existente em relação ao repositório externo de imagens de sensoriamento remoto. Na arquitetura inicial, esse repositório aparecia diretamente como uma fonte externa de dados. No projeto detalhado, *ExternalDataRepository* passou a funcionar como uma fachada de acesso a essa fonte, encapsulando os detalhes necessários para verificar e recuperar novas imagens.

Da mesma forma, o pacote *Models* não corresponde a uma nova camada de processamento. Ele foi criado para reunir entidades utilizadas na comunicação entre os componentes, permitindo representar explicitamente informações como versão, localização dos artefatos e demais metadados associados às imagens, modelos, mapas, zoneamento e alertas.

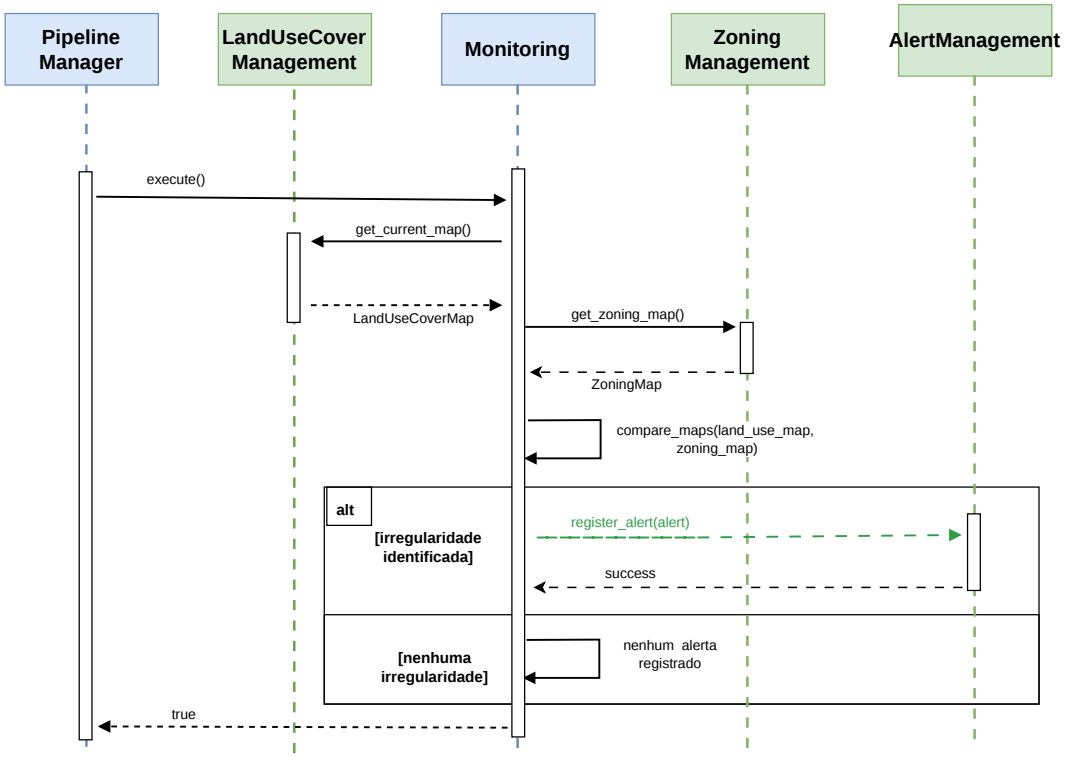


Figura 3. Diagrama de sequência do processo de monitoramento e possível geração de alerta de irregularidade.

A introdução de *PipelineStage* decorreu da necessidade de representar de forma uniforme as diferentes etapas coordenadas pelo *PipelineManager*. Como todas as etapas oferecem uma operação `execute()` e devem informar sucesso ou falha, foi definida uma classe abstrata contendo esse contrato. Dessa forma, o gerenciador não precisa possuir lógica específica para cada implementação concreta e pode percorrer uma coleção de etapas de maneira polimórfica.

Outra alteração foi a criação de *AlertManagement*. Na arquitetura inicial, *Monitoring* comunicava-se diretamente com *ControlledAccess* para disponibilizar os alertas. No detalhamento, essa dependência foi substituída pelo registro dos alertas em *AlertManagement*. Assim, o processamento produz e registra as ocorrências, enquanto a aplicação web as consulta posteriormente, reduzindo o acoplamento entre as camadas.

Os repositórios físicos apresentados anteriormente nas visões de execução e implantação continuam fazendo parte da solução, mesmo quando não são representados como classes no diagrama detalhado. Os componentes responsáveis pelo gerenciamento dos dados encapsulam o acesso a esses recursos. Portanto, *LandUseCoverManagement* controla o armazenamento dos mapas de uso e cobertura, *ZoningManagement* controla o armazenamento do zoneamento, *ModelRepository* controla os artefatos dos modelos e *DataPreprocessedRepository* controla os dados intermediários. A omissão dos arquivos ou dispositivos físicos no diagrama de classes tem como objetivo manter a representação concentrada na estrutura do software, e não eliminar esses elementos da arquitetura.

A implementação dos protótipos também evoluiu durante o processo de detalhamento. Uma primeira versão apresentava dependências mais diretas entre o *PipelineManager* e as etapas concretas e entre *Monitoring* e *ControlledAccess*. A introdução de *PipelineStage* pode ser caracterizada como uma refatoração do tipo *Extract Superclass*, na qual o contrato comum de execução foi explicitado em uma classe abstrata. De maneira semelhante, a introdução de *AlertManagement* corresponde à extração de uma responsabilidade para uma nova classe, aproximando-se da refatoração *Extract Class*. Também foi realizada uma reorganização das classes em pacotes lógicos, isolando as entidades de domínio em *Models* e o acesso à fonte externa em *External*. Essas alterações preservam o comportamento esperado do sistema, mas reduzem o acoplamento e tornam as responsabilidades mais explícitas.

3. Planos de testes

A implementação utilizada para validação do projeto foi desenvolvida em Python. Como o objetivo deste trabalho é verificar a estrutura, as interfaces e as interações estabelecidas pelo projeto detalhado, operações computacionalmente custosas, como treinamento real do modelo, inferência sobre imagens e comparação geoespacial, foram representadas por implementações *dummy*.

Os testes foram implementados utilizando o módulo *unittest*, pertencente à biblioteca padrão do Python. Nesse framework, os casos de teste podem ser organizados em classes derivadas de `unittest.TestCase`, sendo cada método cujo nome inicia com `test_` tratado como um caso de teste independente. O framework oferece métodos de verificação, como `assertTrue()`, `assertFalse()`, `assertEqual()` e `assertIsNone()`, que permitem comparar os resultados produzidos pelas classes com os comportamentos esperados.

Os testes podem ser descobertos e executados automaticamente pelo comando:

```
python3 -m unittest discover -s tests -v
```

A implementação dos casos acompanha o pacote de software entregue. Os testes unitários encontram-se em `tests/test_unit.py`, enquanto os testes de integração encontram-se em `tests/test_integration.py`. As tabelas apresentadas nesta seção descrevem os objetivos e comportamentos esperados de cada caso, enquanto os arquivos Python correspondem à sua implementação executável.

Os testes unitários foram utilizados para verificar individualmente as responsabilidades das classes. Quando necessário, foram utilizados objetos simulados simples para substituir colaboradores externos e permitir que apenas o comportamento da classe sob teste fosse avaliado. Por exemplo, os testes de *PipelineManager* utilizam etapas controladas para verificar se são executadas na ordem configurada, se o fluxo é interrompido após uma falha e se uma exceção inesperada é tratada adequadamente. De maneira semelhante, os testes de *IngestionScheduler* simulam as respostas do repositório externo e do gerenciador do pipeline para verificar seu comportamento nos diferentes cenários.

3.1. (a) Casos de testes unitários

Esses casos verificam as responsabilidades individualmente atribuídas às classes pelo projeto detalhado. Como as operações de inteligência artificial e geoprocessamento ainda são

Tabela 1. Casos de testes unitários implementados.

ID	Classe	Comportamento verificado
TU01	ControlledAccess	Verifica se os alertas armazenados em *AlertManagement* são corretamente retornados pela aplicação web.
TU02	DataPreProcessing	Verifica a obtenção das imagens, execução do pré-processamento *dummy* e armazenamento do resultado.
TU03	Generate	Verifica a criação e disponibilização de um novo mapa associado ao modelo vigente.
TU04	IngestionScheduler	Verifica que o pipeline não é executado quando não existem novos dados.
TU05	IngestionScheduler	Verifica que uma falha informada pelo *PipelineManager* é propagada pelo agendador.
TU06	IngestionScheduler	Verifica que a existência de novos dados provoca a execução do pipeline.
TU07	Monitoring	Verifica que nenhuma ocorrência é registrada quando não são identificadas irregularidades.
TU08	Monitoring	Verifica que a análise falha adequadamente quando não existe mapa disponível.
TU09	Monitoring	Verifica o registro de um *IrregularityAlert* quando uma irregularidade é identificada.
TU10	PipelineManager	Verifica que as etapas são executadas na ordem em que foram configuradas.
TU11	PipelineManager	Verifica que o fluxo é interrompido na primeira etapa que retorna falha e que `failed_stage` é atualizado.
TU12	PipelineManager	Verifica que uma exceção inesperada em uma etapa é tratada como falha e impede a execução das etapas posteriores.
TU13	Training	Verifica que os dados pré-processados são utilizados para produzir uma nova versão do modelo.

simuladas, os testes não procuram demonstrar a correção matemática desses algoritmos, mas verificar se as interfaces, estados e interações especificadas produzem os resultados esperados para as implementações *dummy*.

3.2. (b) Casos de testes de integração

Os testes de integração utilizam os componentes reais da implementação *dummy* conectados da mesma maneira prevista no projeto, permitindo verificar não apenas o comportamento individual das classes, mas também o fluxo de informações entre elas.

Tabela 2. Casos de testes de integração implementados.

ID	Cenário	Comportamento verificado
TI01	Novos dados com irregularidade	Verifica o fluxo completo de pré-processamento, atualização do modelo, geração do mapa, monitoramento e registro de alerta.
TI02	Novos dados sem irregularidade	Verifica a atualização completa do mapa sem criação de alerta.
TI03	Ausência de novos dados	Verifica que o pipeline não é executado e que os estados existentes permanecem inalterados.
TI04	Falha durante nova atualização	Após uma execução bem-sucedida, simula uma falha durante nova atualização e verifica a preservação do mapa, modelo e alertas anteriormente disponíveis.

Os dois primeiros casos exercitam o fluxo completo de atualização em suas duas alternativas de monitoramento. O terceiro verifica a interação entre o *IngestionScheduler* e o restante do pipeline quando nenhuma atualização é necessária. O quarto caso verifica parcialmente o requisito de disponibilidade e confiabilidade, simulando uma falha após uma execução previamente bem-sucedida e confirmando a preservação dos artefatos já disponíveis.

Foram executados 17 casos de teste, sendo 13 unitários e quatro de integração. Todos os casos implementados foram concluídos com sucesso, sem falhas ou erros. Esses resultados demonstram que a implementação *dummy* atende aos cenários contemplados pelo plano de testes, embora não constituam uma validação completa dos algoritmos de inteligência artificial, das operações geoespaciais, da comunicação real com serviços externos ou dos mecanismos definitivos de persistência.

Uma implementação completa deverá ampliar o conjunto de testes para contemplar, entre outros aspectos, a correção dos algoritmos de treinamento e inferência, a persistência e recuperação dos artefatos, a comunicação com o repositório externo, os mecanismos de autenticação e autorização e outras possibilidades de falha durante as diferentes etapas do processamento.

4. Considerações finais

O projeto detalhado da PIMUCS permitiu transformar os componentes definidos na arquitetura inicial em classes, interfaces e relações mais próximas de uma implementação orientada a objetos. A introdução dos pacotes *External* e *Models* explicitou, respectivamente, a fronteira com o sistema externo de sensoriamento remoto e as entidades utilizadas para representar os principais dados da aplicação. A abstração *PipelineStage* estabeleceu uma interface uniforme para as etapas de processamento e permitiu ao *PipelineMana-*

ger coordená-las de maneira polimórfica, reduzindo sua dependência das implementações concretas.

O detalhamento também resultou na separação entre a produção e a consulta dos alertas por meio da classe *AlertManagement*, evitando uma dependência direta entre *Monitoring* e a aplicação web. Os componentes responsáveis pelo gerenciamento de mapas, modelos, zoneamento e dados intermediários permanecem responsáveis pelo controle dos respectivos repositórios físicos, ainda que arquivos, diretórios e mecanismos de armazenamento não apareçam explicitamente no diagrama de classes.

A implementação *dummy* em Python permitiu exercitar as principais interfaces e relações do projeto sem exigir a execução das operações computacionalmente intensivas previstas para o sistema completo. Os testes unitários e de integração implementados com *unittest* verificaram o comportamento dos componentes individualmente e em conjunto, fornecendo evidências de que a estrutura definida no projeto detalhado é compatível com os principais fluxos arquiteturais especificados.

O projeto detalhado não elimina a necessidade de decisões adicionais durante a implementação completa. Persistência durável, versionamento dos artefatos, autenticação, autorização, tratamento abrangente de falhas, treinamento real do modelo, inferência sobre os dados e análise geoespacial permanecem como funcionalidades a serem concretizadas. Entretanto, a estrutura definida fornece uma base modular para essa evolução, mantendo a separação entre processamento, gerenciamento de dados, acesso externo e interação com os usuários.

Na elaboração deste trabalho, os diagramas UML foram construídos utilizando o *diagrams.net (draw.io)*. Também foi utilizado o *ChatGPT*, da OpenAI, como ferramenta de inteligência artificial generativa para apoio à discussão de alternativas de projeto, revisão da consistência entre diagramas, código e testes, revisão textual e elaboração inicial de trechos da implementação *dummy*. As decisões arquiteturais, adaptações do projeto, organização final das classes e validação dos resultados foram realizadas pelo autor.

Referências

Clements, P., Garlan, D., Little, R., Nord, R., and Stafford, J. (2003). Documenting software architectures: views and beyond. In *25th International Conference on Software Engineering, 2003. Proceedings.*, pages 740–741. IEEE.

Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Object Management Group (2017). Omg unified modeling language (omg uml), version 2.5.1. Technical Report formal/2017-12-05, Object Management Group.